Agentic College RAG — Frontend Plan

Clean website-ready chatbot • Antigravity phase prompts

Updated design: lightweight agentic routing, multi-intent planning, evidence-grounded multimodal retrieval, negative-result caching, and transparent fault tolerance.

Build principle: use a small orchestrator with explicit rules and structured model output. Avoid adding a complex agent framework until the simple workflow is proven.

Recommended baseline: React + Vite + TypeScript frontend; Python FastAPI backend; Ollama for local LLM inference; ChromaDB for vector retrieval; SQLite for metadata, audit events, and negative-result cache; OCR for scanned PDFs and screenshots.

1. Frontend goal and visual direction

Build a clean, calm, naturally themed chatbot screen that can later be embedded into the college website. The chat is the main screen, not a sidebar. Keep the interface visually simple and mostly non-interactive beyond essential chat controls. Use a restrained palette (warm off-white, deep navy/forest accent, muted grey), generous whitespace, readable typography, and subtle borders. Avoid dashboards, decorative animations, excessive cards, or a busy admin panel in the public chat view.

2. Main screen layout

Area	Requirements
Header	College name/logo placeholder, “College Assistant” title, small “Answers from official college documents” subtitle. Keep it easy to replace with real branding.
Conversation canvas	Centered main column with welcome message and example prompts such as “What is the Semester 5 fee?”, “When is registration due?”, and “Where is the campus located?”
Message bubbles	Simple user and assistant messages. Assistant responses can include compact source citations and warning notices.
Composer	Text box, send button, attachment icon for screenshot/image, disabled/loading state, and a short file-size/type hint.
Answer evidence	Collapsible or compact source list showing document title, semester/year, page number, version, and excerpt returned by backend.
Status notices	Quiet but clear badges/banners for limited evidence, no answer, OCR unavailable, retrieval unavailable, or temporary LLM error.
Admin entry	Small link or route to a separate admin login. Do not put admin controls into the public chat canvas.

3. Agentic behavior in the UI

- Do not show internal chain-of-thought or detailed routing reasoning. Optionally show simple progress labels: “Understanding your question…”, “Checking college documents…”, “Preparing answer…”.
- For a multi-part query, show one combined answer with numbered or titled sections, not three separate assistant messages.
- If one subtask has evidence and another does not, answer the supported parts and clearly mark the missing part.
- If a query is general conversation, respond naturally without displaying a fake “searching documents” state.
- If a previous no-evidence result is reused, do not mention internal cache details to the student; just return the current safe response.
- Never style an answer as verified merely because it is fluent. The backend’s answer_status and evidence determine the UI.

4. Frontend response contract

Create TypeScript types matching the backend contract. The UI should not independently decide whether a question is general or needs RAG; that decision belongs to the backend orchestrator.

```
type AnswerStatus = | "verified" | "partial" | "no_answer" | "degraded" | "general"; type Evidence =
{ citation_id: string; document_title: string; semester?: string; academic_year?: string; version?:
number; page?: number; snippet: string; }; type ChatResponse = { answer: string; answer_status:
AnswerStatus; warnings: string[]; evidence: Evidence[]; request_id: string; task_summary?: {
completed: number; total: number }; };
```

Suggested endpoints: POST /api/v1/chat for text; POST /api/v1/chat/attachment for screenshot plus question; GET /api/v1/health for minimal health status. Admin routes should be protected and separate.

5. Phase-by-phase Antigravity prompts

Phase 1 — Build the clean main chat screen

```
Build a responsive React + Vite + TypeScript single-page college chatbot frontend. The public chat must be the main screen, not a sidebar. Use a clean, naturally themed design with warm neutral background, deep navy or muted green accents, simple typography, generous whitespace, subtle borders, and no unnecessary animations. Add a replaceable college logo/name, welcome message, three example question chips, chat message area, text composer, send button, and image attachment control. Use mock data only; do not claim it connects to a real backend. Add responsive mobile layout, keyboard accessibility, and a README with exact Windows PowerShell run commands.
```

Phase 2 — Add combined answers, citations, and status states

```
Add typed ChatResponse and Evidence models. Render one assistant answer for each user message, including multi-part answers. Support source citation cards with document title, semester/year, page, version, and snippet. Render answer_status values verified, partial, no_answer, degraded, and general. Show warnings prominently but calmly. Include loading, retry, empty, timeout, and network error states. Use mock responses to demonstrate general greeting, single college query, three-part query, no evidence, and degraded retrieval. Do not invent citations or classify requests in the frontend.
```

Phase 3 — Screenshot upload UX

```
Add image attachment selection for PNG/JPG/JPEG, preview, remove-before-send, file validation, upload progress, and clear errors. Submit the screenshot with the user’s typed question to the attachment endpoint. Do not store question text or image files in localStorage. Clear temporary previews after completion and preserve the typed question if a request fails. Explain that a screenshot helps interpret the question but the answer is checked against official college documents.
```

Phase 4 — Admin pages as a separate area

```
Create a minimal separate admin login and document management route, not a sidebar in the public chat. Add upload fields for title, category, semester, academic year, effective date, and file. Show document processing stages: uploaded, extracting, OCR, embedding, indexed, failed. Add version list, reprocess, and deactivate controls. Use mock API methods until backend is available. The backend must enforce all authorization.
```

Phase 5 — Connect the backend

```
Replace mock API methods with a typed API client. Use VITE_API_BASE_URL for the non-secret backend URL. Implement POST /api/v1/chat and POST /api/v1/chat/attachment. Preserve response fields and display backend warnings accurately. Add timeout handling, retry, request ID details in a small troubleshooting disclosure, and a dev-only mock mode. Never expose Ollama, ChromaDB, admin secrets, or server file paths in the browser.
```

Phase 6 — Final UX and tests

```
Test desktop and mobile layouts, long answers, multiple subtasks, one subtask with no evidence, general chat without retrieval, negative-cache response, OCR failure, ChromaDB outage, Ollama timeout, invalid attachment, and expired admin session. Ensure one user query yields one combined assistant response. Keep the interface clean, accessible, and ready to embed into a college website later.
```

6. Frontend acceptance checklist

- Main chat is the landing screen; no sidebar layout.
- General greetings appear naturally and do not show unnecessary retrieval progress.
- Multi-part queries produce one combined response.
- Sources and warnings are shown only from backend data.
- Screenshot upload supports validation and graceful errors.
- Admin upload screens are separate from the public chat.
- Frontend works with mock mode and then with the real API through configuration.

7. Recommended component structure

```
src/ app/ (router, app shell) components/ (ChatHeader, MessageList, MessageBubble, Composer, AttachmentPreview, EvidenceList, StatusNotice, ExampleQuestions) pages/ (ChatPage, AdminLoginPage, AdminDocumentsPage) api/ (client, chat, documents) types/ (chat, evidence, documents) styles/ (tokens, global, components) tests/
```